Avalanche Validator Lifecycle Model v0.8.0: Differential Execution Specification

Rigorous Implementation Mapping: Two-PSUB cadCAD Architecture, Internal Six-Stage Pipeline, Parallel State Updates, and Invariant Verification

Inputs & Current State X_t

Active Commitments
`C_t (Micro-State)`
stakes, days_left, done, node_key

Out-of-Set Records
`O_t (Eligible Pool)`
out_stakes, out_days ≤ 365d

Global Protocol State
`Z_t = (S_t : supply_navax)`
Cumulative supply counter

Exogenous Inputs U_t

Scheduled Commitments
`Q_t (Bypasses Behaviour)`
Pre-accepted, due on day t

Protocol Configuration
`ψ_prot (Immutable)`
Bounds: S_min=2k, S_max=3M; ACP-285

Historical Benchmark Data
`D (Reference Data)`
D_menu yield menu, empirical tables

Bootstrap Parameters
`θ_r (Run-Specific)`
Refit per run: renewal, return, arrival

Keyed PRNG Streams
`Ξ_(r,t) (Stateless Hash)`
Deterministic common random draws

reads C_t (ending commitments)

reads S_t

reads O_t (eligible out-of-set records)

PSUB 0: Main Daily Lifecycle Transition Block

Executed via single policy function `p_lifecycle` calling `run_stages()` across six linear stages

Day Context & Feedback Signal Evaluation: G(S_t ; D_menu, ψ_prot)

Prepares `Day(t, start, draw, blocks, renewal_gap, entry_gap, settings)`. Evaluates renewal yield gap `g_t^(R)` and entry yield gap `g_t^(E)`.

Yield feedback signals feed directly into Renewal Logit (Stage 1) and Negative-Binomial Arrivals (Stage 3).

Internal Daily Lifecycle Pipeline (run_stages)

Strict execution order: earlier stage outputs pass sequentially via mapping `earlier` to subsequent stages

1. Renewal Stage
renewal_stage
`P(R=1) = σ(x^T β + ρ_R g^(R))`

ended, renews

2. Returns Stage
returns_stage
`h_(j,t) (6 day bins, 8..365d)`

comes_back

3. Arrivals Stage
arrivals_stage
`A_t ~ NegBin(μ_t ≤ 100, φ)`

Participation Candidates N_t = (renews, comes_back, arrivals) pool into Continuation

4. Continuation Stage
`_stake_continuation (single stage)`

Part A: Duration Selection d' ∈ [14, 365]
Empirical tables: 4 previous bins | first_days

Part B: Stake Choice s' ∈ [2k, 3M] AVAX
Hurdle p_(c,b,q) + 10-band log-ratio quantiles

started (s', d')

5. Rewards Stage
rewards_stage (Deterministic)
`I_(m,t) = potential_reward(s', d', S_t)`

issued_navax I_t

6. Assembly Stage
assembly_stage (Deterministic)
`Assemble C_(t+1), O_(t+1); inject Q_t`

injects scheduled Q_t due today (bypasses stages 1-5)

NON POLICY SIGNAL = assembly_stage outputs (stakes, days, out_stakes, issued_navax

State-Update Function Bank: Parallel Updates on Pre-PSUB State X_t

All 24 functions receive (params, substep, history, state=X_t, signal). Executed in parallel, not sequentially.

`s_copy(name) x 8`
Updates: C_(t+1) Commitment Columns
stakes, days_left, done, days, returns, node_key...

`s_copy(name) x 6`
Updates: O_(t+1) Out-of-Set Columns
out_stakes, out_days, out_done, out_last_days...

`s_supply`
Updates: supply_navax (Persistent Z)
state["supply_navax"] + signal["issued_navax"]

`s_count`
Updates: active_validators
len(signal["stakes"]) -> Count of open records

`s_stake`
Updates: active_validator_stake_navax
int(signal["stakes"].sum()) -> Total active stake

`s_out`
Updates: returning_nodes
len(signal["out_stakes"]) -> Count eligible out-of-set

`s_nakamoto`
Updates: nakamoto_third_nodes
Fewest unique node_keys whose stake strictly > 1/3

`s_copy(flows) x 5`
Updates: Intermediate Flows Φ_t
issued_navax, yield_gap_pp, started_stake...

`s_copy(rate)`
Updates: offered_rate_30d
Annualized rate offered for 30d baseline commitment

candidate state X_(t+1)'

PSUB 1: Invariant Validation

Policy `p_invariants` asserts 5 conservation laws

Check 1: Validator Stake Bounds
`2,000 ≤ s_i ≤ 3,000,000 AVAX`

Check 2: Stake Sum Conservation
`active_validator_stake == sum(stakes)`

Check 3: Out-of-Set Max Horizon
`all(out_days ≤ 365)`

Check 4: Protocol Supply Cap
`supply_navax ≤ 720,000,000 AVAX`

Check 5: Solvency Invariant
`active_validator_stake ≤ supply_navax`

Do All 5 Invariants Hold?

False

FAILURE
InvariantError
Halts run

True (Pass)

Validated Next State
`X_(t+1) (Confirmed Sound)`
C_(t+1), O_(t+1), Z_(t+1)

measured from X_(t+1)

measured from Φ_t

Recorded Outputs & Metrics

`Y_t = H(X_(t+1), Φ_t)`

`active_validators` [Snapshot]
N_(t+1) = |C_(t+1)|
Count of active validator records

`active_validator_stake` [Snapshot]
sum_(i ∈ C) s_(i,t+1)
Total stake active on Primary Network

`returning_nodes` [Snapshot]
M_(t+1) = |O_(t+1)|
Count of eligible out-of-set records

`nakamoto_third_nodes` [Snapshot]
min {m : sum S_(j) > 1/3 S_tot}
Decentralization over simulated node keys

`supply_navax` [Cumulative]
S_(t+1) = S_t + I_t
Persistent reward-adjusted supply

`issued_navax` [Flow]
I_t = sum_(m ∈ N) I_(m,t)
Potential rewards from new endogenous starts

`started_stake_navax` [Flow]
sum_(m ∈ N) s'_m
Principal stake committed today

`started_stake_days` [Flow]
sum_(m ∈ N) s'_m * d'_m
Contracted stake-duration flux

`yield_gap_pp` [Signal]
g_t^(R) = offered - benchmark
Renewal yield gap in percentage points

`offered_rate_30d` [Diagnostic]
rate(30d ; S_t, ψ_prot)
Annualized minting rate for baseline stake

SYSTEM FEEDBACK LOOP: Validated State X_(t+1) becomes Input State X_t for D

Visual Grammar & Adapted Notation Legend

- **Cylinder (Blue-Grey):** Persistent Endogenous State Variable (C_t, O_t, Z_t)
- **Inverted Triangle (Cream):** Exogenous Input / Scenario Parameter (Q_t, D, ψ_prot)
- **Diamond (Muted Pink):** Stochastic Behavioural Policy / Decision Function
- **Pentagon (Sage Green):** Mechanism / State-Update Operation *(Adapted convention)*
- **Double-Border Box (Red):** Formal Invariant Assertion (Raises InvariantError on failure)
- **Rectangle (Grey):** Recorded Output Metric / Observable Vector Y_t
- **Solid Arrow:** State/Signal Dataflow | **Dashed Blue Line:** Inter-timestep Feedback Loop

**Adapted Visual Convention: In BlockScience/Polkadot engineering specifications, pentagons frequently denote control functions. Here, pentagons der*

Traceability Matrix & Verified Model Boundaries (model-v0.8.0 at b18a4a2)

CadCAD Architecture & Execution Semantics

- **Exact PSUB Count:** Exactly 2 blocks per daily step (checked_blocks in engine.py:89).
- **Policy Function:** p_lifecycle (fitted_lifecycle.py:286) executes run_stages.
- **Stage Orchestration:** run_stages executes stages sequentially; earlier passes to later.
- **Parallel State Updates:** All 24 update functions read pre-PSUB state and common signal.
- **Supply Accumulation:** s_supply computes state["supply"] + signal["issued"] (not sequential).
- **Validation Timing:** p_invariants runs in PSUB 1 immediately after all state updates.
- **Error Raising:** Raises InvariantError on failure, terminating run without corruption.

Behavioural Processes & Calibration Laws

- **Renewal Logit:** node_record disabled (held at 0.0) in released v0.8.0 configuration.
- **Return Hazard:** Piecewise logistic hazard across 6 day-out bins (8..365d); no yield term.
- **Arrivals Count:** Negative Binomial count model; daily mean capped at 100.0 nodes/day.
- **Continuation Duration:** Empirical duration table (4 bins) for renewals/returns; first_days for new.
- **StakeChoice Law:** Two-part hurdle (p) + 10 log-spaced stake bands; 50-obs fallback threshold.
- **Stake Uncertainty:** Fixed point estimate across runs (stake hurdle not resampled).
- **PRNG Innovations:** Stateless keyed draws (draw.uniforms) tie randomness to entity identity.

Verified Model Boundaries & Scope Exclusions

- **Constant Minimum Stake:** Released model strictly enforces S_min = 2,000 AVAX for all d in [14, 365].
- **Proposed Admission Frontier:** S_min(d) = S_floor + (S_short - S_floor)η(d)^p is NOT active/enforced.
- **Uptime / Slashing:** No uptime tracking, liveness checks, or slashing exist in v0.8.0.
- **Gross Potential Rewards:** issued_navax reflects gross potential rewards without uptime forfeiture.
- **Identity Representation:** Concentration measured over simulated node_keys, not corporate entities.
- **Scheduled Commitments:** Exogenous inputs (Q_t) bypass duration and stake choice mechanisms.
- **Delegations & Operators:** Excluded from v0.8.0; scheduled for v0.8.1 delegation lifecycle.